

Experiment No. 4

Aim

Socket programming for transport layer packet (UDP server)

Theory

At the Transport Layer of the OSI model, **UDP (User Datagram Protocol)** is a connectionless, unreliable, and lightweight protocol. Unlike TCP, UDP does not establish a virtual circuit or perform a three-way handshake prior to data transfer. Each transmitted data unit is handled as an independent **datagram**.

UDP Socket Programming on the server side listens for incoming datagrams from any client without maintaining state or active connections.

UDP Server Execution Steps:

1. **Socket Creation (socket()):** The server initializes a socket using the address family AF_INET (IPv4) and socket type SOCK_DGRAM (UDP datagram).
2. **Binding (bind()):** The socket is bound to a specific local IP address and port number using bind(). This designates the entry point for incoming client datagrams.
3. **Receiving Data (recvfrom()):** The server blocks and waits for datagrams. The recvfrom() system call captures the client's payload alongside the client's IP address and port number.
4. **Sending Data (sendto()):** The server sends a response back to the client using sendto(), explicitly targeting the address structure captured during reception.
5. **Closing Socket (close()):** Releases the socket descriptor resources when the process completes.

Name : Tanishq Ambagade

PRN: 24070521284

```
lab-03-25@lab-03-25-OptiPlex-3280-A10: $ vim server_udp284.c
lab-03-25@lab-03-25-OptiPlex-3280-A10: $ gcc server_udp284.c -o server
lab-03-25@lab-03-25-OptiPlex-3280-A10: $ ./server
UDP Server is running on port 8080...
Message from Client: hi

lab-03-25@lab-03-25-OptiPlex-3280-A10: $ ./server
UDP Server is running on port 8080...
Message from Client: tanishq 284
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8080
int main()
{
    int server_socket;
    char buffer[1024];
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    // Create UDP socket
    server_socket = socket(AF_INET, SOCK_DGRAM, 0);
    if (server_socket < 0)
    {
        perror("Socket creation failed");
        return 1;
    }
    // Configure server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    // Bind socket
    if (bind(server_socket, (struct sockaddr *)&server_addr,
        sizeof(server_addr)) < 0)
    {
        perror("Bind failed");
        close(server_socket);
        return 1;
    }
    printf("UDP Server is running on port %d...\n", PORT);
    // Receive message from client
    int n = recvfrom(server_socket, buffer, sizeof(buffer) - 1, 0,
        (struct sockaddr *)&client_addr, &client_len);
    if (n < 0)
    {
        perror("Receive failed");
        close(server_socket);
        return 1;
    }
    buffer[n] = '\0';
    printf("Message from client: %s\n", buffer);
    // Send response to client
    char *response = "Hello, I am UDP Server";
    sendto(server_socket, response, strlen(response), 0,
        (struct sockaddr *)&client_addr, client_len);
    // Close socket
    close(server_socket);
    return 0;
}

```

Conclusion

The UDP Server socket was successfully implemented at the Transport Layer. By utilizing SOCK_DGRAM with recvfrom() and sendto(), the server successfully bound to its designated port and processed incoming connectionless datagrams without establishing a prior network handshake.